

Patient Health Monitoring Data Pipeline using AWS

BUSINESS SCENARIO

A hospital receives patient health monitoring files every hour from:

- ICU monitoring systems
- Wearable health devices
- Hospital management systems
- Emergency care systems

The hospital wants to:

- Store incoming patient data securely
- Validate health records automatically
- Detect corrupted files
- Clean and transform patient data
- Catalog metadata automatically
- Store processed healthcare data
- Separate bad patient records
- Maintain production-ready ETL workflows

Objective

Build a healthcare ETL pipeline that processes patient monitoring data securely.

PROJECT GOALS

build a system that:

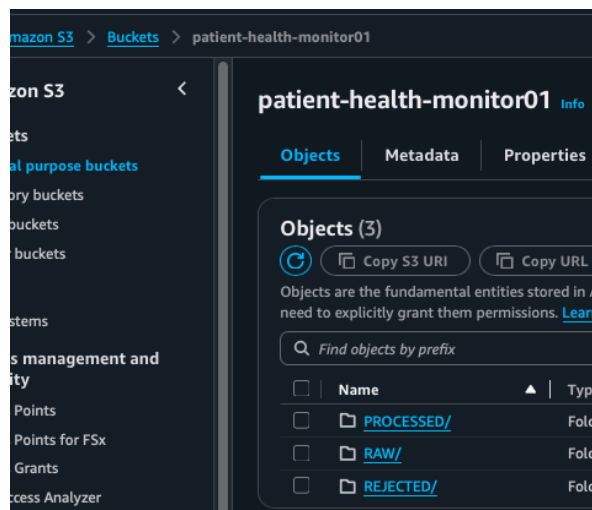
- Ingests patient data
- Validates incoming files
- Cleans medical records
- Detects invalid patient readings
- Stores transformed healthcare data
- Handles schema changes

HEALTH METRICS TO TRACK

Metric	Example
Heart Rate	72 BPM
Blood Pressure	120/80
Oxygen Level	98%
Temperature	98.6 F
Respiration Rate	16/min

S3 storage – patient s-health-monitor-bucket

raw/
 Vitals/
 Patient/
 Device/
Processed/
 Vitals/
 Patients/
 Reports /
Rejected/



CREATION OF IAM IMPLEMENTATION

Glue IAM Role
Lambda IAM Role

SAMPLE HEALTHCARE DATA

FILE 1 — patient_vitals.csv

```
patient_id,timestamp,heart_rate,blood_pressure,oxygen_level,temperature
P1001,2025-01-10 08:00:00,72,120/80,98,98.6
P1002,2025-01-10 08:05:00,95,140/90,96,99.1
P1003,2025-01-10 08:10:00,45,90/60,88,101.2
P1004,2025-01-10 08:15:00,,130/85,97,98.4
P1005,2025-01-10 08:20:00,110,150/95,,100.1
P1005,2025-01-10 08:20:00,110,150/95,,100.1
```

FILE 2 — patient_details.csv

patient_id,name,age,gender,city,disease
P1001,Rahul Sharma,45,Male,Delhi,Diabetes
P1002,Priya Verma,52,Female,Mumbai,Hypertension
P1003,Amit Singh,60,Male,Pune,Heart Disease
P1004,Neha Kapoor,29,Female,Delhi,Asthma
P1005,Rohit Jain,48,Male,Jaipur,Diabetes

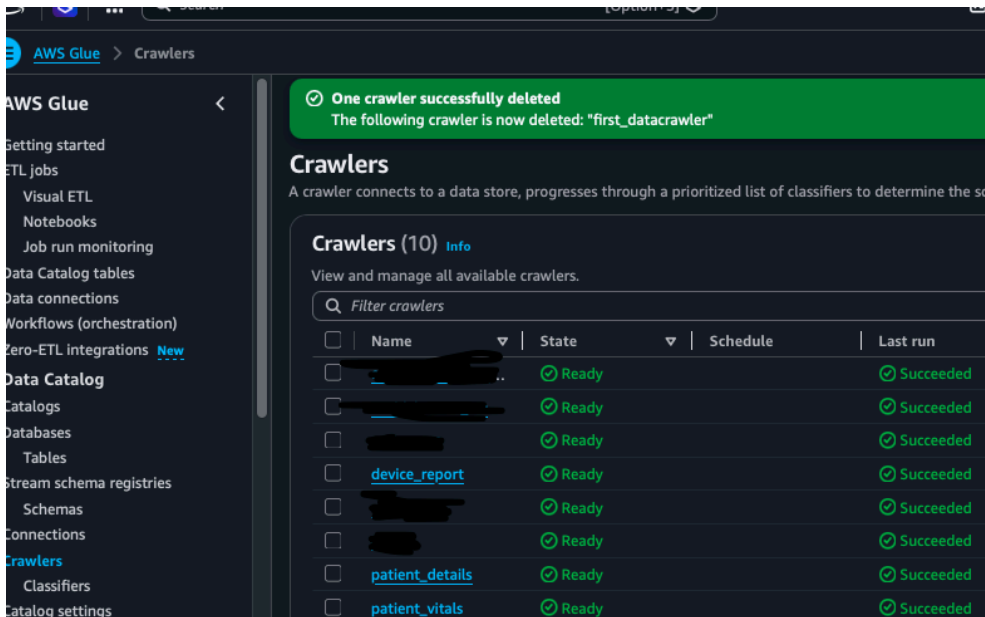
FILE 3 — device_logs.csv

device_id,patient_id,device_type,status,last_sync
D101,P1001,SmartWatch,Active,2025-01-10 08:00:00
D102,P1002,Oximeter,Inactive,2025-01-10 07:50:00
D103,P1003,HeartMonitor,Active,2025-01-10 08:05:00

DETECT FOLLOWING:

- | Issue | Example |
- | ----- | ----- |
- | Null values | heart_rate missing |
- | Duplicate records | P1005 repeated |
- | Critical health readings | oxygen_level = 88 |
- | Missing patient metrics | oxygen_level blank |
- | Schema changes | new column added |

AWS CRAWLER TO READ HEALTH CARE BUCKET RAW DATA
Automatically discover healthcare schema.



DATABASE AND TABLE AUTOMATICALLY SCANNED FROM RAW BUCKET

Getting started

Jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Getting started

Jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

You can now create Apache Iceberg tables in the AWS Glue Data Catalog. To learn more, visit the documentation.

Create table

00010300

Tables

A table is the metadata definition that represents your data, including its schema. A table can be used as a source or target in a job definition.

Tables (4)

Last updated (UTC)
June 15, 2026 at 16:48:33

Delete

Add tables using crawler

Add table

Choose catalog

039914330453

Choose database

health_monitor_db

Q Filter tables

☐	Name	Database	Location	Classificati...	Deprecated	View data
☐	device_reports	health_monitor_db	s3://patient-health-monitor01/RAW/device_reports/	CSV	-	Table data
☐	patient_details	health_monitor_db	s3://patient-health-monitor01/RAW/patient_details/	CSV	-	Table data
☐	patient_health_rep	health_monitor_db	s3://patient-health-monitor01/crawleroutput	-	-	Table data
☐	patient_vitals	health_monitor_db	s3://patient-health-monitor01/RAW/patient_vitals/	CSV	-	Table data

MAIN HEALTHCARE ETL PIPELINE USING PYSPARK

Getting started

ETL Jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

Zero-ETL integrations

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows (orchestration)

Blueprints

Getting started

ETL Jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Zero-ETL integrations

Data Catalog

Catalogs

Databases

Tables

Stream schema registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

Zero-ETL integrations

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows (orchestration)

Blueprints

healthcare etl pipeline

Script

Job details

Runs

Data quality

Schedules

Version Control

Script

Info

```
1 import sys
2 from awsglue.transforms import *
3 from awsglue.utils import getResolvedOptions
4 from pyspark.context import SparkContext
5 from awsglue.context import GlueContext
6 from awsglue.job import Job
7 from pyspark.sql import functions as F
8
9 ## @params: [JOB_NAME]
10 args = getResolvedOptions(sys.argv, ['JOB_NAME'])
11
12 sc = SparkContext()
13 glueContext = GlueContext(sc)
14 spark = glueContext.spark_session
15 job = Job(glueContext)
16 job.init(args['JOB_NAME'], args)
17
18 database = "health_monitor_db"
19
20 vitals_df = glueContext.create_dynamic_frame.from_catalog(
21     database=database,
22     table_name="patient_vitals"
23 ).toDF()
24
25 patients_df = glueContext.create_dynamic_frame.from_catalog(
26     database=database,
27     table_name="patient_details"
28 ).toDF()
29
30 devices_df = glueContext.create_dynamic_frame.from_catalog(
31     database=database,
32     table_name="device_reports"
33 ).toDF()
34
35 # Remove duplicate records
36 vitals_df = vitals_df.dropDuplicates()
37
38
39
40
```

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows

(orchestration)

Zero-ETL integrations

New

Data Catalog

Catalogs

Databases

Tables

Stream schema

registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

Zero-ETL integrations

New

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows

(orchestration)

Blueprints

Security configurations

healthcare etl pipeline

Script

Job details

Runs

Data quality

Schedules

Version Control

Script

Info

```
37
38 # Remove duplicate records
39 vitals_df = vitals_df.dropDuplicates()
40
41 # Remove records with missing patient_id
42 vitals_df = vitals_df.filter(F.col("patient_id").isNotNull())
43
44 # Convert datatypes
45 vitals_df = (
46     vitals_df
47     .withColumn("heart_rate", F.col("heart_rate").cast("int"))
48     .withColumn("oxygen_level", F.col("oxygen_level").cast("int"))
49     .withColumn("temperature", F.col("temperature").cast("double"))
50     .withColumn("timestamp", F.to_timestamp("timestamp"))
51 )
52
53 # Handle null readings
54 vitals_df = vitals_df.fillna({
55     "heart_rate": 0,
56     "oxygen_level": 0,
57     "temperature": 0
58 })
59
60 # Remove incorrect values
61 vitals_df = vitals_df.filter(
62     (F.col("heart_rate") >= 0) &
63     (F.col("heart_rate") <= 250) &
64     (F.col("oxygen_level") >= 0) &
65     (F.col("oxygen_level") <= 100) &
66     (F.col("temperature") >= 90) &
67     (F.col("temperature") <= 110)
68 )
69
70
71 # Remove duplicate patient records
72 patients_df = patients_df.dropDuplicates(["patient_id"])
73
74 # Remove records with missing patient_id
75 patients_df = patients_df.filter(
76     F.col("patient_id").isNotNull()
77 )
```

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows

(orchestration)

Zero-ETL integrations

[New](#)

Data Catalog

Catalogs

Databases

Tables

Stream schema

registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and

ETL

Zero-ETL integrations

[New](#)

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data

detection

Record Matching

Triggers

Workflows

(orchestration)

Blueprints

Security configurations

healthcare etl pipeline

Script

Job details

Runs

Data quality

Schedule

Script Info

```
74
75 # Remove records with missing patient_id
76 patients_df = patients_df.filter(
77     F.col("patient_id").isNotNull()
78 )
79
80 # Handle missing patient information
81 patients_df = patients_df.fillna({
82     "name": "Unknown",
83     "disease": "Unknown",
84     "city": "Unknown",
85     "gender": "Unknown"
86 })
87
88 # Remove incorrect ages
89 patients_df = patients_df.filter(
90     (F.col("age") >= 0) &
91     (F.col("age") <= 120)
92 )
93
94
95 # Remove duplicate device records
96 devices_df = devices_df.dropDuplicates()
97
98 # Remove records with missing patient_id
99 devices_df = devices_df.filter(
100     F.col("patient_id").isNotNull()
101 )
102
103 # Handle missing device information
104 devices_df = devices_df.fillna({
105     "device_type": "Unknown",
106     "status": "Unknown"
107 })
108
109 # Convert timestamp
110 devices_df = devices_df.withColumn(
111     "last_sync",
112     F.to_timestamp("last_sync")
113 )
114
```

AWS Glue

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows

(orchestration)

Zero-ETL integrations

New

Data Catalog

Catalogs

Databases

Tables

Stream schema

registries

Schemas

Connections

Crawlers

Classifiers

Catalog settings

Data Integration and ETL

Zero-ETL integrations

New

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Interactive Sessions

Data classification tools

Sensitive data detection

Record Matching

Triggers

Workflows

(orchestration)

Blueprints

Security configurations

healthcare etl pipeline

ScriptJob detailsRunsData qualitySchedulesVersion Control

ScriptInfo

```
110 # Convert timestamp
111 devices_df = devices_df.withColumn(
112     "last_sync",
113     F.to_timestamp("last_sync")
114 )
115
116 # Keep valid statuses only
117 devices_df = devices_df.filter(
118     F.col("status").isin("Active", "Inactive", "Unknown")
119 )
120
121 vitals_df = vitals_df.withColumn(
122     "health_status",
123     F.when(
124         (F.col("heart_rate") == 0) |
125         (F.col("oxygen_level") == 0) |
126         (F.col("temperature") == 0),
127         "Missing Vitals"
128     )
129     .when(F.col("oxygen_level") < 90, "Critical")
130     .when(F.col("temperature") > 100, "Fever")
131     .when(F.col("heart_rate") > 100, "High Risk")
132     .otherwise("Normal")
133 )
134
135 vitals_df = vitals_df.withColumn(
136     "alert_flag",
137     F.when(F.col("health_status") == "Normal", "N").otherwise("Y")
138 )
139
140 vitals_df = vitals_df.withColumn(
141     "monitoring_date",
142     F.to_date("timestamp")
143 )
144
145 vitals_df = vitals_df.withColumn(
146     "monitoring_hour",
147     F.hour("timestamp")
148 )
149
150
```

AWS Glue

healthcare etl pipeline

Script Job details Runs Data quality Schedules Version Control

```
145 vitals_df = vitals_df.withColumn(  
146     "monitoring_hour",  
147     F.hour("timestamp")  
148 )  
149  
150  
151 final_df = vitals_df.alias("v") \  
152     .join(patients_df.alias("p"), "patient_id", "left") \  
153     .join(devices_df.alias("d"), "patient_id", "left")  
154  
155 # FINAL OUTPUT  
156  
157 report_df = final_df.select(  
158     F.col("patient_id"),  
159     F.col("name").alias("patient_name"),  
160     F.col("disease"),  
161     F.col("heart_rate"),  
162     F.col("oxygen_level"),  
163     F.col("temperature"),  
164     F.col("health_status"),  
165     F.col("device_type"),  
166     F.col("monitoring_date")  
167 )  
168  
169 # WRITE TO S3  
170 output_path = "s3://patient-health-monitor01/PROCESSED/finalreport/"  
171  
172 report_df.write \  
173     .mode("overwrite") \  
174     .format("parquet") \  
175     .save(output_path)  
176  
177 report_df.write \  
178     .mode("overwrite") \  
179     .format("parquet") \  
180     .option("path", "s3://patient-health-monitor01/crawleroutput/") \  
181     .saveAsTable("health_monitor_db.patient_health_report")  
182  
183 job.commit()  
184  
185
```

Python Ln 85, Col 24 0 Errors: 0 | 0 Warnings: 0

PROCESSED DATA IN PARQUET FORMAT

Amazon S3 > Buckets > patient-health-monitor01 > PROCESSED/ > finalreport/

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

finalreport/

Objects Properties

Objects (1)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 Inventory](#) to get a list of all objects in your bucket. For others permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified
☐	part-00000-799371db-a4de-4f78-b60f-13f66149d1c2-c000.snappy.parquet	parquet	June 8, 2026, 17:06:18 (UTC-05:00)

OUTPUT

Amazon Athena > Query editor

Data

Data source

AwsDataCatalog

Catalog

None

Database

health_monitor_db

Tables and views

Create

Filter tables and views

Tables (4)

device_reports

patient_details

patient_health_report

patient_vitals

Views (0)

Query 1 : X | Query 2 : X | Query 3 : X | Query 4 : X | Query 5 : X | Query 6 : X | Query 7 : X

1 SELECT * FROM "AwsDataCatalog"."health_monitor_db"."patient_health_report" limit 10;

SQL Ln 1, Col 1

Run again

Explain

Cancel

Clear

Create

Reuse query results

up to 60 minutes ago

Query results

Query stats

Completed

Time in queue: 106 ms

Run time: 354 ms

Data scanned: 0.63 KB

Results (5)

Copy

Download results CSV

Search rows

#	patient_id	patient_name	disease	heart_rate	oxygen_level	temperature	health_status	device_type	monitoring_d
1	P1001	Rahul Sharma	Diabetes	72	98	98.6	Normal	SmartWatch	2025-01-10
2	P1002	Priya Verma	Hypertension	95	96	99.1	Normal	Oximeter	2025-01-10
3	P1003	Amit Singh	Heart Disease	45	88	101.2	Critical	HeartMonitor	2025-01-10
4	P1004	Neha Kapoor	Asthma	0	97	98.4	Missing Vitals		2025-01-10
5	P1005	Rohit Jain	Diabetes	110	0	100.1	Missing Vitals		2025-01-10